

For information on the proper approach using catkin, start here <http://answers.ros.org/question/52013/catkin-and-eclipse/>.

4. Catkin and Python

For me, the above procedure didn't generate a .pydevproject file, like make eclipse-project ever did. Clicking Set as PyDev Project would create a config but without any Paths, so coding would be a hassle.

Workaround: From within the package you want to program run:

- 1 python \$(rospack find mk)/make_pydev_project.py

Now copy the created file .pydevproject (which has all dependency package paths included) to /build and import your catkin-project into eclipse or set it as PyDev Project if already imported.

5. catkin tools

With the new catkin_tools, there are few changed from the Catkin-y method described above. To generate eclipse-project you need to execute:

- 1 \$ catkin build --force-cmake -G"Eclipse CDT4 - Unix Makefiles"

to generate the .project files for each package and then run: the following script

- 01 \$ ROOT=\$PWD
- 02 \$ cd build
- 03 \$ for PROJECT in `find \$PWD -name .project`; do
- 04
- 05 DIR=`dirname \$PROJECT`
- 06 echo \$DIR
- 07 cd \$DIR
- 08 awk -f \$(rospack find mk)/eclipse.awk .project > .project_with_env && mv .project_with_env .project
- 09
- 10 \$ done
- 11 \$ cd \$ROOT

To debug use the following command and you can mention the name of the package to configure that specific project for debug instead of the entire workspace. Remember to run the script to modify .project to pass the current shell environment into the make process in Eclipse.

- 1 \$ catkin build --force-cmake -G"Eclipse CDT4 - Unix Makefiles" - DCMAKE_BUILD_TYPE=Debug